Physics-Informed LLM Latency Prediction:
Critical Analysis of the Analytical Roofline Model vs. Direct Operational Modeling

Pre-Execution Scheduling & The Token Count Prerequisite

In production LLM serving, latency predictors must operate **strictly before execution** using pre-launch parameters: cluster parallelism (`tp`, `pp`, `dp`, `num_gpus`) and request budgets (`batch_size`, `output_tokens`). Post-execution telemetry (measured duration, energy, power) is strictly excluded.

The Token Prerequisite (Imai et al., Section A.4): The analytical Roofline Model (RLM) calculates hardware ceilings from input and output tokens:

$$T_{\text{roofline}} = T_{\text{prefill}}(\text{input_tokens}) + \text{output_tokens} \times T_{\text{decode_step}}(\text{batch_size}) \tag{1}$$

While prompt token count is known pre-execution via fast tokenization, output length is dynamic. As acknowledged by Imai et al. (Appendix A.4, *Limitation*): *"In practical application, the generation length is not predetermined... To accurately model inference latency in real world scenarios, we must consider predicting the generation length based on the input prompt."*

Core Thesis Critique: Is RLM Calculation Necessary If Tokens Are Known?:

- **When Token / Task Context is Known (RLM is Redundant)**: If token counts or task context are known prior to execution, our evaluations demonstrate that **calculating the analytical Roofline model is empirically unnecessary**. Standard tree ensembles (RF/GB) trained directly on operational parameters already reach near-perfect accuracy ($R^2 = 0.9997$), with RLM adding +0.0% **improvement**. Non-linear models easily learn hardware scaling directly from empirical data without manual hardware profiling.
- **When Prompt Context is Omitted (RLM Acts as a Surrogate)**: If prompt context is omitted (e.g., privacy/zero-data retention prevents payload inspection), naive ML collapses ($R^2 \approx 0.25$). Here, RLM serves as a vital physics proxy, leaping RF accuracy to 0.9997 (**+74.4% gain**, **99.96% MSE reduction**).

Table 1: Cross-Validation on Qwen3-14B: Known Operational Features vs. Roofline-Enhanced Regimes.

Prediction Method	Base R^2	Base MSE	RLM R^2	RLM MSE	R^2 Gain (Δ)
Regime 1: Omitted Prompt Context (Parallelism Known, Prompt Unknown) — Roofline is Vital					
Linear Regression (LR)	0.3005	224573.9	**0.8844**	**37920.9**	**+58.4%**
Random Forest (RF)	0.2553	236925.7	**0.9997**	**87.8**	**+74.4%**
Gradient Boosting (GB)	0.2494	238558.9	**0.9995**	**163.7**	**+75.0%**
Regime 2: Known Token / Task Context (Direct Operational Modeling) — Roofline is Redundant for Trees					
Linear Regression (LR)	0.7879	69037.9	**0.8882**	**36782.7**	**+10.0%**
Random Forest (RF)	**0.9997**	86.5	**0.9997**	87.8	**+0.0%**
Gradient Boosting (GB)	**0.9997**	113.8	**0.9997**	163.7	**+0.0%**

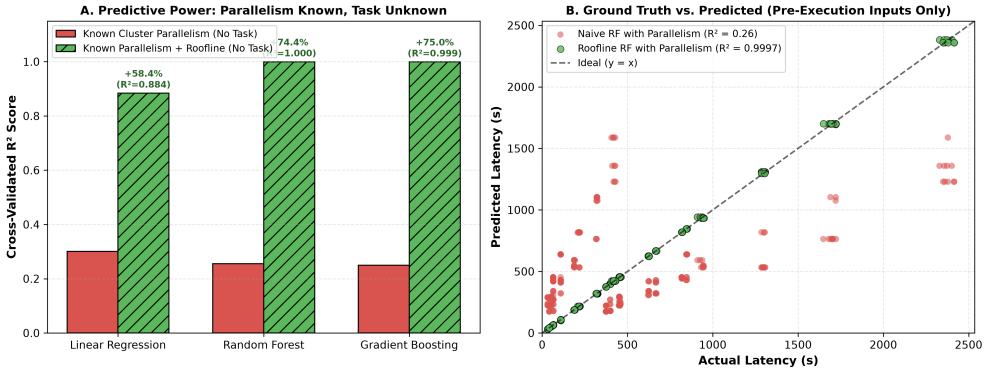


Figure 1: **(A)** R^2 comparison: Naive models fail without prompt context; Roofline restores accuracy. **(B)** Predicted vs. Actual latency: With Roofline, predictions align perfectly with ground-truth latency.

Key Conclusions & Need for Broader Empirical Validation

- **Direct Operational Modeling Suffices**: When pre-execution token counts are available, direct empirical regression on operational parameters achieves optimal accuracy ($R^2 = 0.9997$), rendering complex Roofline modeling redundant.
- **Niche for RLM**: The Roofline model is justified primarily when restricted to linear models (LR) or when prompt context is inaccessible.
- **Scope & Future Validation**: To conclusively prove this thesis across production environments, broader testing is required across **additional model architectures** (e.g., Mixture-of-Experts MoE, 70B+ scales) and **diverse real-world datasets** (e.g., dynamic multi-turn chats, arbitrary RAG prompt lengths), beyond fixed benchmark grids.